Design: Energetic Mesh Smoothing on General Geometries via Analytic Surface Constraints

Alejandro Mota

July 8, 2026

Abstract

Energetic mesh smoothing (EMS) drives each element toward an ideal reference shape by solving a quasi-static problem in which nodes relax to minimize a stored-energy functional. Interior nodes move freely; boundary nodes must remain on the domain boundary, sliding within it. Today this boundary restriction is imposed with axis-aligned Dirichlet conditions on the faces of a cube, which works only because a cube face's outward normal coincides with a Cartesian basis vector. This note generalizes the boundary restriction to any boundary that can be described analytically as a level set $g(\boldsymbol{x}) = 0$, using the same symbolic-expression pipeline already used for initial and boundary conditions. It states the constraint model (tangential slip with a return-to-surface correction, and a dimensional hierarchy for faces, edges, and vertices), two enforcement routes (a soft penalty and an exact local-frame constraint), the robustness guards, and a test plan that includes exact reduction to the current cube behavior. Because a Genesis/Exodus file carries no analytic geometry—a curved boundary is stored only as a soup of flat element facets, verified below—the note treats two interchangeable surface sources feeding a common constraint: the user-supplied analytic level set (exact but requires a formula) and the mesh's own boundary facets (general but only as smooth as the input mesh).

1 Problem statement

EMS treats the mesh as a hyperelastic body. For each element a target ("ideal") reference configuration is constructed from a size criterion (equal-volume, average-edge-length, `max`, or a user size field; see `create_smooth_reference` in `model.jl`), and a quasi-static solve lets the nodal positions $\boldsymbol{x}$ relax so that elements adopt shapes as close as possible to their targets, minimizing a stored-energy functional $W(\boldsymbol{x})$.

Interior nodes are free in $\mathbb{R}^3$. Boundary nodes are *not*: a node on the domain boundary $\partial\Omega$ must stay on $\partial\Omega$, or the smoothing would change the geometry. Physically:

- a node on a boundary *face* may slide anywhere on that face (2 degrees of freedom);
- a node on a boundary *edge* (where two faces meet) may slide only along the edge (1 degree of freedom);
- a node on a boundary *vertex* must stay put (0 degrees of freedom).

For a cube, all of this is imposed trivially. Each face is axis-aligned, so "stay on the face $x = \text{const}$" is exactly "hold the x component fixed, leave y, z free." The current EMS examples do precisely that with six Dirichlet conditions, one Cartesian component per face (`examples/ems/awful-cube/awful-cube.yaml`): edges and vertices need no special treatment because a node belonging to two or three faces inherits two or three component constraints automatically.

This is a coincidence of alignment. The moment the boundary is not axis-aligned—a cylinder's lateral surface, a sphere, a fillet—there is no Cartesian component to fix, and the current mechanism

cannot express the constraint at all. The goal of this note is to lift EMS to any boundary whose faces admit an analytic description, so that the method can be explored on realistic geometries.

2 Geometric constraint model

2.1 A boundary face as a level set

Assume each smooth boundary face is given as the zero level set of a scalar field,

$$g(\boldsymbol{x}) = 0, \qquad \boldsymbol{x} \in \mathbb{R}^3, \tag{1}$$

with g smooth and $\nabla g \neq \mathbf{0}$ on the face (a regular level set). The outward normal direction is

$$\boldsymbol{n}(\boldsymbol{x}) = \frac{\nabla g(\boldsymbol{x})}{\|\nabla g(\boldsymbol{x})\|}. \tag{2}$$

A node constrained to this face may move only tangentially. To first order in an increment $\delta\boldsymbol{x}$,

$$g(\boldsymbol{x} + \delta\boldsymbol{x}) \approx g(\boldsymbol{x}) + \nabla g(\boldsymbol{x}) \cdot \delta\boldsymbol{x}, \tag{3}$$

so requiring the node to remain on the face, $g(\boldsymbol{x} + \delta\boldsymbol{x}) = 0$, gives the linearized constraint

$$\nabla g(\boldsymbol{x}) \cdot \delta\boldsymbol{x} \;=\; -\, g(\boldsymbol{x}). \tag{4}$$

The right-hand side is a *return-to-surface* (drift) correction: it vanishes when the node lies exactly on the face and otherwise pulls it back onto the face as the quasi-static iteration proceeds. On a curved face, where a tangential step necessarily leaves the surface at second order, this term is what keeps the node on the true geometry rather than on the tangent plane.

Dividing by $\|\nabla g\|$ writes the same constraint in terms of the unit normal and the signed distance-like residual,

$$\boldsymbol{n}(\boldsymbol{x}) \cdot \delta\boldsymbol{x} \;=\; -\,\frac{g(\boldsymbol{x})}{\|\nabla g(\boldsymbol{x})\|}. \tag{5}$$

When the face is axis-aligned, $\nabla g = \pm\boldsymbol{e}_k$, and (5) reduces to "fix the k-th Cartesian component"—exactly the present cube behavior. The general case is the classical *inclined (skew) roller support*: the node is free to slide in the plane tangent to the surface and pinned in the normal direction.

2.2 Edges and vertices: the dimensional hierarchy

A boundary edge is the intersection of two faces, and a vertex the intersection of three. A node is therefore constrained by *all* the faces it belongs to. If it satisfies m independent level sets $g_1, \ldots, g_m$ (with $m \in \{1, 2, 3\}$), it must satisfy (4) for each:

$$\nabla g_i(\boldsymbol{x}) \cdot \delta\boldsymbol{x} = -\, g_i(\boldsymbol{x}), \qquad i = 1, \ldots, m. \tag{6}$$

The constraint gradients $\boldsymbol{a}_i = \nabla g_i(\boldsymbol{x})$ span the *constrained subspace* $\mathcal{C} = \operatorname{span}\{\boldsymbol{a}_1, \ldots, \boldsymbol{a}_m\}$, of dimension m; the admissible motions are its orthogonal complement, the *tangent subspace* $\mathcal{T} = \mathcal{C}^{\perp}$, of dimension $3 - m$:

feature	level sets m	free DOF $3 - m$	admissible motion
face	1	2	slide in the tangent plane
edge	2	1	slide along the edge tangent
vertex	3	0	pinned

This recovers, with no special cases, the cube behavior in which membership in one/two/three faces leaves two/one/zero free components. The user declares which level sets apply to which boundary node set; the feature dimension is simply the number of level sets listed (or an explicit `surface`/`curve`/`vertex` tag).

3 Surface representation and evaluation

The constraint model of Section 2 needs only two quantities at each constrained node: the signed surface residual $g(\boldsymbol{x})$ and the unit normal $\boldsymbol{n} = \nabla g / \|\nabla g\|$ (one such pair per incident face at an edge or vertex). Everything downstream is agnostic to *how* they are produced. There are two sources, and which applies is dictated by what the mesh file actually carries.

3.1 What a Genesis/Exodus file stores

A Genesis file records no analytic geometry. Cubit-generated primitives confirm it: a curved boundary exported as a side set is stored as two integer arrays, `elem_ss` (element numbers) and `side_ss` (local face index on each element), with unit distribution factors—a list of (element, local-face) pairs, that is, a soup of flat element facets. For a meshed sphere the "spherical" side set is just the outward hexahedral faces (`elem_ss` the 24 surface elements, `side_ss` all equal to the same local face); nothing records that it is a sphere, its radius, or its normals. The curvature lives *only* in the nodal coordinates, and node sets are plain node lists. Two consequences follow: the true smooth surface must be supplied by the user if it is wanted, because the file cannot provide it; yet the mesh already carries a usable—if piecewise-flat—representation of the boundary in the side-set facets themselves.

3.2 Analytic level sets

Norma already compiles analytic expressions of (t, x, y, z) for initial and boundary conditions and for the EMS size field: the string is parsed with `Meta.parse`, turned into a `Symbolics` expression, and compiled to a fast closure by `build_function` (see `SolidMechanicsDirichletBoundaryCondition` in `boundary_conditions.jl` and `create_size_field` in `model.jl`). The surface field g is compiled by the identical path.

The key economy is that the gradient need not be supplied or approximated. From the symbolic g,

$$\nabla g = \texttt{Symbolics.gradient}(g,\, [x, y, z]) \tag{7}$$

is obtained *exactly and automatically* and compiled the same way; the Hessian $\nabla^2 g$ is available identically if a full penalty tangent is wanted. The user writes only the surface, e.g. `"x^2 + y^2 - R^2"` for a cylinder of radius R about the z-axis; the normal field follows without finite differencing.

Proposed YAML schema. Mirroring the initial/boundary-condition style, each constrained node set lists one or more surfaces; the count sets the feature dimension.

```
model:
  type: mesh smoothing
  smooth reference: max
  boundary surfaces:
    - node set: lateral          # face: 1 level set, 2 free DOF
      surfaces: ["x^2 + y^2 - R^2"]
```

```
- node set: top_rim          # edge: 2 level sets, 1 free DOF
  surfaces: ["x^2 + y^2 - R^2", "z - H"]
- node set: mount_corner     # vertex: 3 level sets, pinned
  surfaces: ["x", "y", "z - H"]
```

Constants such as `R` and `H` are substituted into the string as they already are for other expressions. A node set with no listed surface is left free (interior-like), and any Cartesian face can still be written as a linear level set (e.g. `"x"`), so the cube case is a special case of the schema.

3.3 Discrete boundary facets

When no analytic form is available—an imported or otherwise formless geometry—the boundary side set supplies the surface directly. Constrain each boundary node to the union of its side-set facets in the reference configuration by closest-point projection: the foot point is the drift target, the facet normal is $\boldsymbol{n}$, and the signed distance to the facet is the residual g in (5). Norma already has this machinery— `closest_point_projection` and the containing-facet scan used by the overlap-energy blending—so the discrete evaluator is largely an assembly of existing parts and requires no new user input beyond naming the boundary side set.

Faces, edges, and vertices are recovered by a feature-angle test on the facet normals incident to a node: a single cluster of normals is a smooth face ($m = 1$, project onto the boundary); two clusters mark an edge ($m = 2$, constrain to the ridge where the groups meet); three or more mark a corner ($m = 3$, pinned). This is the discrete analogue of the level-set count of Section 2.

Trade-off. The discrete surface is only as smooth as the input faceting: nodes slide on the piecewise-flat boundary and cannot recover a surface smoother than the mesh already resolves. For quality-driven smoothing—which redistributes nodes without refining the boundary—that is usually the intent, and it is the only option when the geometry has no closed form. The analytic route is preferable when the smooth surface is known and sub-faceting fidelity on the boundary matters. The two are combinable: analytic where a formula is known (primitive faces), discrete elsewhere.

4 Enforcement

The EMS examples solve with matrix-free descent (steepest descent or L-BFGS; see `examples/ems/awful-cube/`), which uses the gradient of W and, for Newton, its Hessian. Both enforcement routes below fit that machinery; they differ in invasiveness and in how exactly the surface is honored. Recall the solver reduces the system to the free DOF: for Newton, $\boldsymbol{A} = \texttt{hessian}[\text{free}, \text{free}]$, $\boldsymbol{b} = \texttt{gradient}[\text{free}]$, step $= -\boldsymbol{A}^{-1}\boldsymbol{b}$; for matrix-free descent, the step is taken along the (projected) negative gradient over the free DOF (`solver.jl`).

4.1 Route A — penalty (prototype)

Augment the smoothing energy with a quadratic penalty on each constrained node's surface residuals,

$$P(\boldsymbol{x}) = \sum_{\text{nodes}} \sum_{i=1}^{m} \tfrac{1}{2}\,\kappa_i\, g_i(\boldsymbol{x})^2. \tag{8}$$

Its contributions to the residual (gradient) and tangent (Hessian) at a constrained node are

$$\frac{\partial P}{\partial \boldsymbol{x}} = \sum_i \kappa_i \, g_i \, \nabla g_i, \tag{9}$$

$$\frac{\partial^2 P}{\partial \boldsymbol{x}^2} = \sum_i \kappa_i \big(\nabla g_i \, \nabla g_i^\top + g_i \, \nabla^2 g_i \big) \;\approx\; \sum_i \kappa_i \, \nabla g_i \, \nabla g_i^\top, \tag{10}$$

the last a Gauss–Newton approximation (drop the $g_i \nabla^2 g_i$ term, which vanishes as $g_i \to 0$). Each term is a rank-one operator $\kappa_i \, \nabla g_i \nabla g_i^\top$ that penalizes motion *along the normal* while leaving tangential motion governed by W—a soft version of the exact roller. For the matrix-free solvers EMS actually uses, only (9) is needed; it is added to `solver.gradient` at the node's three DOF after `evaluate`. No `free_dofs` manipulation and no change of basis are required.

Trade-offs. Trivial to implement and the fastest way to exercise the level-set and gradient plumbing on a curved mesh. But the surface is satisfied only approximately—the residual scales like (smoothing force)/κ—and large κ ill-conditions the system. κ_i must be scaled to the element stiffness and mesh size (e.g. $\kappa \sim c\,\mu/h^2$). It is a derisking and validation tool, not the production mechanism.

4.2 Route B — exact local-frame (inclined roller)

Enforce (6) exactly by a change of basis at each constrained node. Orthonormalize the constraint gradients $\{\boldsymbol{a}_1, \ldots, \boldsymbol{a}_m\}$ (Gram–Schmidt / thin QR) into $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_m$ spanning $\mathcal{C}$, and complete them to an orthonormal frame

$$\boldsymbol{Q} = [\underbrace{\boldsymbol{q}_1 \cdots \boldsymbol{q}_m}_{\text{normal}} \mid \underbrace{\boldsymbol{q}_{m+1} \cdots \boldsymbol{q}_3}_{\text{tangent}}] \in \mathbb{R}^{3\times 3}, \tag{11}$$

whose first m columns span the constrained (normal) directions and last $3-m$ span the tangent. In the rotated coordinates $\widetilde{\delta \boldsymbol{x}} = \boldsymbol{Q}^\top \delta \boldsymbol{x}$, the tangential components are free and the normal components are prescribed by the drift correction. For a single face ($m = 1$), $\widetilde{\delta x}_1 = -g_1/\|\boldsymbol{a}_1\|$ from (5); for $m > 1$ the m normal components follow from the small linear system $\boldsymbol{A}_{\mathcal{C}} \, \widetilde{\delta \boldsymbol{x}}_{1:m} = -\boldsymbol{g}$ with $(\boldsymbol{A}_{\mathcal{C}})_{ij} = \boldsymbol{a}_i \cdot \boldsymbol{q}_j$.

Two ways to carry this through the solver.

- *Matrix-free descent (the EMS default).* The exact constraint is a *projected gradient with return to surface*: (i) project the node's gradient onto the tangent subspace, $\boldsymbol{g}^{\tan} = (\boldsymbol{I} - \sum_i \boldsymbol{q}_i \boldsymbol{q}_i^\top)\, \boldsymbol{g}^{\text{node}}$, so the descent direction has no normal component; (ii) after the step, restore the node to the surface by a few Newton iterations of $g_i(\boldsymbol{x}) = 0$ along the normal (a closest-point projection). This keeps every constrained node on the manifold to tolerance at each iteration and needs no assembled operator—the least invasive exact route and the natural fit for steepest descent / L-BFGS.

- *Newton (`HessianMinimizer`).* Apply the nodal rotation to the assembled system: left-multiply the constrained node's rows of the residual and Hessian by $\boldsymbol{Q}^\top$ and right-multiply its columns by $\boldsymbol{Q}$ (a congruence transform that preserves symmetry), mark the m normal DOF as fixed in `free_dofs` with prescribed increment equal to the drift correction, leave $3-m$ tangential DOF free, solve the reduced system, and rotate the increment back, $\delta \boldsymbol{x} = \boldsymbol{Q}\, \widetilde{\delta \boldsymbol{x}}$.

Both variants reduce *exactly* to the present behavior when the faces are axis-aligned: then $\boldsymbol{Q}$ is a signed permutation, the projection zeroes a Cartesian component, and the fixed normal

DOF is a Cartesian component—the cube DBCs, recovered as a special case. This is the production mechanism: variationally consistent, surface-exact up to the return-to-surface tolerance, and geometry-agnostic.

5 Robustness and degeneracy guards

Vanishing gradient. $\|\nabla g\| \to 0$ marks a critical point of the level set (e.g. the center of a sphere), where the normal (2) is undefined. Abort with a diagnostic, consistent with the degeneracy policy already adopted for the overlap-energy blending (reject, do not regularize).

Node off the declared surface. The node set is asserted by the user to lie on the surface. If $|g(\boldsymbol{x}_0)|$ exceeds a tolerance scaled by the local element size at setup, warn (or abort): the mesh does not match the declared geometry, which is almost always an input error.

Rank-deficient feature. At an edge or vertex the gradients $\{\boldsymbol{a}_i\}$ must be linearly independent; two nearly parallel surfaces make the frame system $\boldsymbol{A}_{\mathcal{C}}$ ill-conditioned (a near-tangential intersection). Detect via the smallest singular value and abort as a degenerate feature.

Return-to-surface convergence. On strongly curved faces the drift correction is only first order; cap the per-iteration normal correction and, if the closest-point Newton fails to converge, fall back to a bisection along the normal so a node never leaves its surface.

6 Test plan

1. **Reduction to the cube (regression).** Re-express the six cube faces of `awful-cube` as linear level sets (x, $x-L$, y, $\dots$) and verify EMS produces the same smoothed mesh (to solver tolerance) as the current axis-aligned Dirichlet conditions. This pins backward compatibility: the general path must contain the special one.
2. **Cylinder (the motivating case).** Use an existing cylindrical mesh (e.g. `examples/ahead/.../torsion/torsion.g`), perturb the interior nodes, constrain the lateral boundary to $g = x^2 + y^2 - R^2 = 0$, and check that (a) the smoothing energy decreases monotonically, (b) every boundary node satisfies $|g| <$ tol after the solve, and (c) boundary nodes moved tangentially (the surface did not shrink or bulge). Include the top/bottom rims as edges ($\{x^2 + y^2 - R^2,\ z \mp H\}$) to exercise $m = 2$.
3. **Penalty vs. exact.** On the cylinder, confirm the penalty surface residual scales as $1/\kappa$ while the exact route holds $|g|$ at the return-to-surface tolerance independent of any penalty parameter.
4. **Sphere / quarter-annulus.** A doubly curved face ($x^2 + y^2 + z^2 - R^2$) and a mixed straight/curved boundary to check that the normal field and the dimensional hierarchy behave at genuinely non-axis-aligned edges and vertices.
5. **Discrete evaluator (formula-free).** Smooth the same sphere using only its boundary sideset facets (Section 3.3)—no analytic surface—and confirm the result matches the analytic-sphere run to the faceting scale, and that the feature-angle test tags the smooth surface as a face (no spurious edges). This validates the general-geometry path on a geometry whose analytic form is, in this case, known for comparison.

7 Recommendation and phasing

Phase 1. Level-set input (Section 3) plus the penalty enforcement (Section 4.1) and the cylinder test. This derisks the input parsing, the `Symbolics` gradient path, and the membership/normal evaluation with the smallest possible code change, and produces visible "nodes hug the surface" behavior early.

Phase 2. The exact local-frame constraint (Section 4.2)—projected-gradient/return-to-surface for the matrix-free solvers first, then the Newton nodal rotation—and the reduce-to-cube regression. This is the production mechanism.

Phase 3. The discrete-facet evaluator (Section 3.3) as the general-geometry surface source, sharing Phase 2's enforcement and adding the feature-angle detection of edges and vertices. This is the path for imported meshes with no closed form, and it is what the Genesis representation makes the natural default; validate on the sphere against its analytic counterpart.

Phase 4 (optional). Edges and vertices ($m > 1$) end-to-end for the analytic route, plus refinements: a symmetric smoothstep or harmonic weighting if a C^1 ramp is ever wanted, and caching of the compiled $g, \nabla g$ per node set alongside the existing size-field closure.

The construction reuses machinery already in the tree—the symbolic-expression compiler, `Symbolics.gradient`, the `free_dofs` reduced solve, and the `compute_rotation_matrix` helper in `boundary_conditions.jl`—and degrades exactly to the current cube behavior, so the present EMS examples remain a regression test of the general path.